Then to add a view to `businessViews`, you'd do this:

```
increment businessViews, '/products/fade-o-meter/enterprise-
edition'
```

Now `increment` is modifying the `businessViews` object. To be clear, you should avoid state where possible, as chapter 6 will help you to understand. If you *really* need state, then it's best to contain the state and have the object that owns the state be responsible for changing it. With that in mind, here's a new version of the page views code.

Listing 4.7. Page views revisited

```
views =
  clear: ->
    @pages = {}
  increment: (key) ->
    @pages ?= {}
    @pages[key] ?= 0
    @pages[key] = @pages[key] + 1
  total: ->
    sum = 0
    for own page, count of @pages
      sum = sum + count
    sum
```

@ is the this reference to the receiving object.

Initialize the pages property on the specific object on the first call to increment. It's important that a property that keeps state does not live on the prototype. The prototype should contain common behavior.

Now create two view counters:

```
businessViews = Object.create views
personalViews = Object.create views
```

If you increment the value for just one URL on that object 100 times, then the total is 100. But the other object maintains its own state separately. It isn't affected by increments to the first object:

```
for i in [1..100] by 1
  businessViews.increment '/product-details/2454'

businessViews.total()
# 100

personalViews.total()
# 0
```